

Error Handling in Large Systems

Exercises, labs, and review questions for this chapter.

Convert panic-based logic into typed errors, separate domain and infrastructure failures, and add context to async propagation

EXERCISE 1 WARM-UP COMPREHENSION

Choose Option, Result, panic, or process abort from the caller's ability to act

OBJECTIVE

Practice separating ordinary absence, recoverable failure, and broken invariants before one error type grows into a junk drawer.

STARTER PROMPT

Classify four events: an optional request header is missing, a database call times out, a user attempts an invalid state transition, and an internal invariant says two states cannot coexist but now both do.

- Which event is local absence best modeled as Option first?
- Which events are recoverable and should become Result with a typed error?
- Which event is an invariant break more honestly modeled as panic or crash policy?
- Where would you translate the typed error if the next boundary is HTTP, AMQP, or FFI?

ACCEPTANCE CRITERIA

- You distinguish Option, Result, and panic in operational terms rather than by style.
- You classify at least one domain failure separately from one infrastructure failure.
- You explain one translation step at a boundary instead of assuming the same contract survives everywhere unchanged.

HINTS

- Ask first what the caller can still do next.
- If the process state is broken, the question may no longer be 'how do I return an error?'

EXERCISE 2 CODE READING

Separate domain and infrastructure errors in one service path

OBJECTIVE

Read one mixed error surface and decide which variants belong to the domain layer, which belong to adapters, and which belong to top-level translation.

STARTER PROMPT

A billing service currently returns `anyhow::Result<>` from deep inside the domain model and mixes `invalid transition`, `SQL timeout`, and `missing queue channel` into one free-form message chain.

- Which failures belong in a domain error enum and why?
- Which failures belong in infrastructure or adapter errors and why?

- Where should anyhow still remain useful after the separation?
- Which caller actions become easier once the surface is typed again?

ACCEPTANCE CRITERIA

- You move at least one business-rule failure out of the infrastructure bucket.
- You keep anyhow or an equivalent aggregated error layer only where pattern matching has stopped being useful.
- You name at least one caller action that typed separation enables, such as retry, reject, or dead-letter.

HINTS

- A typed contract is most useful where the next layer still branches on it.
- A service shell can still aggregate rich context after lower layers stay typed.

EXERCISE 3 IMPLEMENTATION**Convert panic-based parsing into typed errors****OBJECTIVE**

Replace unwrap-driven control flow with a small typed contract that distinguishes missing input from invalid input.

STARTER PROMPT

Refactor `parse_port` so it returns

`Result<u16, ConfigError>` with separate `MissingPort` and `InvalidPort` variants instead of panicking.

- Keep the error type small and specific.
- Use `ok_or` or an equivalent explicit missing-input branch.
- Use `map_err` or an equivalent parse-failure branch.
- Return the parsed port directly on success instead of logging inside the helper.

ACCEPTANCE CRITERIA

- The parser no longer panics for routine invalid input.
- Missing and malformed input are distinguishable in the return type.
- The runnable lab prints the expected missing, invalid, and success cases.

HINTS

- This is the same contract repair many real config loaders need first.
- A small enum is often enough to make the whole caller path calmer.

EXERCISE 4 DEBUGGING OR REFACTORING**Add context to async task propagation****OBJECTIVE**

Repair a spawned async path so join failure, inner failure, and boundary context stay separate and reviewable.

STARTER PROMPT

You inherit

`tokio::spawn(async move { do_work(job).await })`

followed by `.await?`, but the resulting error text never says whether the task failed to join or the inner operation failed after joining.

- Where should you add context before the spawn boundary and after the await?

- Why is `JoinHandle<Result<T, E>>` a two-layer error surface instead of one?
- Which fields belong in the context message: job ID, queue name, attempt, resource name?
- Which failures in this path are retryable, and where would you record that policy so callers do not infer it from message text?
- When should cancellation be modeled distinctly from ordinary inner failure?

ACCEPTANCE CRITERIA

- You describe the join layer and the inner error layer separately.
- You add at least one useful context message at the operation layer and one at the task layer.
- You explain one case where cancellation or shutdown should not be collapsed into a generic timeout string.

HINTS

- A task panic is not the same incident as a typed adapter or domain failure.
- Context should name the operation, not just repeat the same low-level noun again.

EXERCISE 5 DEBUGGING OR REFACTORING**Translate a Rust error contract across FFI without leaking internals****OBJECTIVE**

Replace a foreign-facing Rust-shaped error surface with one the other runtime can actually consume and stabilize.

STARTER PROMPT

A native plugin boundary currently exposes

```
pub extern "C" fn run(ptr: *const u8, len: usize) -> anyhow::Result<u64>.
```

- Which public boundary types should replace `anyhow::Result` here?
- How will null input, invalid UTF-8, and internal failure be distinguished?
- Which error detail should stay inside the Rust-side log or trace instead of escaping directly over the ABI?
- How would you test the status boundary with a tiny native harness?

ACCEPTANCE CRITERIA

- You translate the boundary into status codes, out parameters, or another explicit ABI-safe contract.
- You distinguish at least two foreign-visible failure classes clearly.
- You keep richer Rust internals behind the wrapper instead of leaking them as the public ABI story.

HINTS

- Another runtime cannot safely consume your internal `anyhow` chain as if it were a stable public type.
- Boring ABI contracts are a feature, not a failure of imagination.

EXERCISE 6 DESIGN OR PRODUCTION SCENARIO**Design one error contract for HTTP, queue workers, and a native plugin****OBJECTIVE****STARTER PROMPT**

Make
retryability,
caller
action,

You are designing

HTTP request -> domain check -> async worker -> database -> native risk plugin -> publish outcome,
and you want another team to operate the failure modes without reverse-engineering your code.

- Which failures should be visible as domain errors, infrastructure errors, and top-level service errors?

boundary

- Which failures are retryable, which are terminal, and where should that policy live?

translation,

- Which boundary should log the error, and which boundary should only translate and return it?

and logging

- What trace or log fields belong on every failure path to make the incident reconstructable later?

discipline

explicit

across

several

large-

system

seams at

once.

ACCEPTANCE CRITERIA

- You define at least one domain error, one infrastructure error, and one translated service or transport contract.
- You make retryable versus terminal classification explicit instead of relying on message text.
- You identify one boundary that logs and another that only translates to avoid duplicate noise.
- You mention at least two observability fields such as request ID, task ID, attempt, queue, or plugin call name.

HINTS

- A large-system error model is easier to operate when retry policy and logging policy are visible in different places on purpose.
- The same failure should not need five log lines before an operator can tell what actually happened.

Runnable lab

Example 41-L. `typed_error_lab.rs` — *Runnable lab · Replace panic-based parsing with typed errors*

```
#[derive(Debug, PartialEq, Eq)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.unwrap();
    Ok(text.parse:::<u16>().unwrap())
}

fn main() {
    println!("missing = {:?}" , parse_port(None));
    println!("bad = {:?}" , parse_port(Some("oops")));
    println!("ok = {:?}" , parse_port(Some("8080")));
}
```

OUTPUT

```
missing = Err(MissingPort)
bad = Err(InvalidPort)
ok = Ok(8080)
```

Review questions

- When is `Option` enough, and when should the boundary promote absence into `Result`?
- Why are domain errors and infrastructure errors worth keeping distinct even if both eventually become one HTTP response or one queue outcome?
- What is the practical split between `thiserror` and `anyhow` in a large Rust system?

- Why does a spawned task usually create a two-layer error surface rather than one?
- Why should a boundary usually log once rather than at every layer that propagates the same error upward?